

Measuring Validator and Sandboxed-Execution Coverage for LLM-Generated NetOps Artifacts: A Paired Confirmatory Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (study_id f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e) that measured the marginal detection benefit of adding sandboxed emulation to a deterministic validator for LLM-generated intent→configuration artifacts. Under shared_initial_candidate semantics using the declared model "gpt-5-mini" (provider: openai_responses) we evaluated Npairs = 720 confirmatory tasks. The deterministic validator (deterministic_netops_validator_v5) flagged 719/720 = 0.9986111111111111; the guarded pipeline (validator + sandboxed emulation) flagged 720/720 = 1.0. Paired difference (guarded - baseline) = 0.001388888888888884; exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI (10000 resamples, seed = 1729) = [0.0, 0.004166666666666667]. Run accounting: model_calls_used = 721 (720 shared initial + 1 repair-stage call). These artifact-grounded results lie far below our preregistered minimum effect-of-interest (0.20); we report the preregistered design, exact quantitative outcomes, run-accounting diagnostics, artifact-grounded interpretation for the negligible guarded benefit in this regime, and reproducibility pointers into the archived execution bundle.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent into device configuration and NetOps workflows. Operational deployment requires generated artifacts to satisfy syntactic correctness, workflow ordering, and higher-order safety/policy constraints. A common mitigation architecture is generate→validate→(repair)→execute, sometimes augmented by sandboxed emulation to reveal runtime-only failures that static validators miss. Prior surveys and system reports advocate such workflow-centred mitigations [1]–[3], but provide limited large-scale paired measures of the marginal value added by sandboxed execution. Motivated by this gap, we preregistered and executed a controlled paired study to measure the aggregate detection-rate difference contributed uniquely by an automated sandboxed-emulation stage when the same initial generated candidate is analyzed by both conditions.

Research question and contributions. Our preregistered research question was: for synthetic intent→configuration tasks under shared_initial_candidate semantics, what is the paired detection-rate difference (guarded - baseline) between (A) a deterministic validator-only pipeline and (B) the same deterministic validator followed by automated sandboxed emulation? Contributions: (1) a machine-readable preregistration and faithful autonomous execution; (2) an artifact-grounded

paired analysis on Npairs = 720 with complete accounting; (3) an artifact-grounded interpretation explaining a near-zero marginal effect; (4) concise reproducibility pointers into the archived bundle and prescriptive boundary conditions where guarded stages are likely to matter.

II. RELATED WORK

Workflow-centred mitigations and verification tools. Recent survey and architectural reports document the practical trend of combining LLM generation with deterministic validators, emulators, and repair loops to raise operational assurance for NetOps artifacts [1]. These works emphasize that correctness in intent→configuration pipelines is multidimensional: syntactic parsing, command-ordering/workflow constraints, invariants (e.g., keep-one-uplink-up), and dynamic runtime properties may each require different V&V primitives. Tool-focused case studies highlight the evolution and practical limits of static verifiers: mature configuration-analysis systems have progressively expanded rule-sets and topology-aware checks but still exhibit gaps for runtime-dependent behaviors and stateful sequences [2]. That evolution shows why researchers and operators often layer an emulator or sandbox after static validation: emulation can exercise temporal behaviors, device state transitions, and interactions that are difficult to capture exhaustively as static rules.

Empirical and prototype evidence. Prototype systems that integrate generation with verification and execution-harnesses report heterogeneous gains: in some task regimes a verifier alone catches most dangerous errors, while in others emulation uncovers subtle runtime failures or environment-specific side effects that static rules miss [1], [3]. The practical implication drawn across this literature is that the marginal utility of a guarded (validator+emulation) pipeline depends on (a) validator coverage relative to the task distribution, (b) the fidelity of the emulator, and (c) the distributional properties of the input intents (e.g., constrained, parser-validated intents versus noisy natural-language requests). These prior empirical and system-level observations motivated a paired experimental design: by reusing the identical initial generated candidate across conditions (shared_initial_candidate semantics) we isolate only downstream guarded-stage effects (emulation observations or permitted repair invocations) rather than conflating them with generation variability.

Gaps and evaluation needs. While surveys and system reports argue for generate→validate→repair and for emulator-assisted validation as best practice for high-assurance NetOps, they do not provide large-scale, preregistered paired estimates of the marginal detection benefit under controlled generation semantics. The literature also documents important open questions that our study was designed to probe: how much additional coverage does sandboxed emulation provide when a strong deterministic validator already targets the same workflow patterns, and which task regimes (adversarial inputs, cross-vendor CLI heterogeneity, or stateful multi-step procedures) are most likely to reveal nontrivial guarded-stage benefits? By situating our paired confirmatory design amid these prior findings, we provide an artifact-grounded quantitative datapoint addressing one narrow slice of the broader evaluation agenda advocated in [1]–[3].

III. METHODOLOGY

Preregistration and confirmatory design. We preregistered a paired-binary confirmatory design (study_id f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e). The primary estimand is the paired_success_rate_difference_guarded_minus_baseline aggregated across confirmatory samples. The preregistered sample plan targeted Npairs = 720 (planned episodes = 1440) with a minimum effect-of-interest of 0.20 (20 percentage points) for power planning. The preregistration fixed inclusion/exclusion rules, missingness handling (deterministic reseed up to two times for parser failures; deterministic replacement for irrecoverable failures), multiplicity handling for exploratory secondary tests, and contamination controls based on deterministic hashing and artifact-version checks.

Task generation, topology templates, and vendor abstractions. Confirmatory items were generated deterministically from the intent_configuration_repair_v1 task family using a deterministic task generator. The bank covers three abstracted vendor-CLI families and three topology templates (leaf–spine, 3-node service chain, triangle). Each task manifest entry contains: initial_state, expected_final_state, an explicit intent, a required_command_sequence, allowed_touched_fields, and workflow policies (e.g., make-before-break, must-be-down-for-fields, minimum_active_in_groups). Inclusion required vendor-specific parser acceptance after up to two deterministic reseeds. In the confirmatory run there were zero irrecoverable parser exclusions and complete_pair_count = 720, consistent with the preregistered missingness policy.

Model, sampling semantics, and shared_initial_candidate enforcement. The declared/requested model was recorded as "gpt-5-mini" (provider: openai_responses) and the model configuration archived (requested_max_output_tokens = 2000; the client did not set an explicit temperature parameter—recorded as null in the model configuration). Important preregistered clarifications: (a) temperature recorded as null indicates the client supplied no explicit temperature value; it does not imply provider-side determinism or temperature=0, and it does not alter the shared_initial_candidate contract; (b) shared_initial_candidate semantics were enforced per the

preregistration: for each pair we obtained exactly one sampled initial candidate (a single provider call) and that identical candidate was presented to both downstream conditions (baseline and guarded). This design isolates downstream guarded-stage effects by removing generation variability as a source of between-condition discordance.

Baseline validator specification. The baseline validator (deterministic_netops_validator_v5) implements deterministic, rule-based intent-constraint validation. Its core checks (applied to a normalized candidate) include: vendor-CLI syntactic parsing; per-command parsing and canonicalization; sequence-order checks against required_command_sequence; atomicity/workflow constraints (e.g., make-before-break, all_down_before_any_field_change); protected-interface invariants; group-level activity constraints (minimum/maximum active interfaces within specified groups); and final-state conformance tests (expected_final_state). The validator returns a binary detection flag (detected/not-detected) under the preregistered scoring rule full_intent_constraint_validation and emits a validator_trace object (validation_before/after, parsed_command_count, required_command_count, violation_codes).

Guarded pipeline and sandboxed-emulation harness. The guarded pipeline runs the baseline validator and, irrespective of the validator outcome, then executes the candidate in an automated sandboxed-emulation harness that instantiates the declared topology template and executes normalized CLI commands against vendor-abstracted virtual devices. The emulation harness records runtime observations relevant to operational safety: interface admin transitions and timing, transient downtime and concurrent-interface states, MTU/VLAN application effects, and observable connectivity or reachability consequences at the topology level. Emulation results are translated into a set of runtime-observation rules that can trigger a guarded detection if they reveal policy or safety violations not encoded in the static validator rule set. The emulation harness is intentionally conservative in mapping observations to detections; its purpose in this study is to reveal runtime-only misconfigurations that the baseline validator might miss.

Repair policy and run accounting. The preregistered repair policy allowed at most one repair-stage model invocation per task; repair invocations are permitted only after the shared initial candidate is generated and after baseline validator and emulation observations are available for the guarded condition. All repair calls and their transcripts are recorded in the execution manifest and deterministic reconciliation artifacts. Provider-call auditing for the confirmatory execution records model_calls_used = 721 with stage_counts showing shared_initial_generation = 720 and repair = 1; thus only a single repair-stage call occurred across the entire confirmatory set. Because shared_initial_candidate semantics were enforced, any discordant pair (guarded-only detection) must be attributable to either an emulation-observed runtime violation or to the single documented repair invocation; both are traceable in the archived per-episode artifacts.

Inclusion, determinism, and contamination controls. The

execution used deterministic artifact hashing, generator-seed recording, and explicit binary/configuration-version checks before confirmatory runs. Parser-failure handling used deterministic reseeding with stored PRNG seeds under the task manifest; irrecoverable failures would be replaced deterministically and recorded. Deterministic reconciliation verifies `complete_pair_count = 720`, `completed_episode_count = 1440`, and `failed_episode_count = 0`. Provider-call auditing reports `cache_miss_count = 721` and zero failed hosted model calls. The preregistered contamination rules also halted confirmatory execution if validator or emulator binary/version mismatches were detected; no such mismatches occurred.

Analysis procedures and pre-registered inferential methods. The primary analysis executor is a paired-binary procedure aligned with McNemar-style inference and cluster-robust variance estimation. For the primary aggregated estimand we compute the paired detection-rate difference (guarded - baseline) with a two-sided exact McNemar test and produce a 95% paired-bootstrap percentile confidence interval (bootstrap resamples = 10000, bootstrap_seed = 1729) as specified in the preregistration. Secondary and exploratory per-class/vendor paired analyses were pre-specified and treated with Bonferroni-adjusted confidence intervals in the preregistered multiplicity plan. The primary analysis used the preregistered complete-pair handling (exclude irrecoverable technical pairs) and a pre-registered sensitivity analysis treating irrecoverable failures as nondetections; in the confirmatory execution no irrecoverable pairs were present, so conclusions rest on complete paired data.

Artifact-locator guidance for independent verification. All per-episode raw scoring and trace artifacts were archived. The canonical input-results file is `execution/raw_results.jsonl` (recorded in the execution manifest). Independent verifiers can locate any discordant pair by searching that file for an entry where `baseline_score = 0` and `guarded_score = 1`; `deterministic_reconciliation.json` in the analysis artifacts links pair accounting and provider_call_audit entries and identifies the repair-stage call(s) associated with discordant outcomes. Summary paired counts and contingency outputs are archived as `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv`. The analysis used these archived artifacts as the direct input to the paired-binary executor; the numeric outputs reproduced in Results are verbatim from `analysis/results.json`.

Design rationale and trade-offs. We chose `shared_initial_candidate` semantics to produce a conservative, direct estimate of guarded-stage marginal contributions: by reusing the same sampled initial candidate for both conditions, any observed discordance can only arise downstream of generation (emulation observation or repair). This isolates the guarded-stage value but intentionally omits any potential benefit that could accrue from independent re-generation in the guarded condition (e.g., prompting-based repair prior to validator execution). Our inclusion policy (parser-validated synthetic tasks) reduces noise and ensures task executability but also concentrates the task bank where a strong validator

can achieve high coverage; this design choice is central to interpreting the near-zero aggregate effect observed in the confirmatory run.

IV. RESULTS

Primary confirmatory counts and test statistics (artifact-grounded). The preregistered paired analysis produced the following exact contingency counts (`analysis/paired_contingency_table.csv`, sha256 efe6e8e7016fa843a8226e911dbde829e943903d719101da8d956b3ff899133c): `n11 = 719` (detected by both), `n10 = 1` (guarded-only), `n01 = 0` (baseline-only), `n00 = 0` (neither). Marginal detection rates are: `baseline = 719/720 = 0.9986111111111111` and `guarded = 720/720 = 1.0`. The paired estimate (guarded - baseline) = 0.0013888888888888884. The exact McNemar two-sided test reported `p = 1.0`. Paired-bootstrap percentile 95% CI (10000 resamples, bootstrap_seed = 1729) = [0.0, 0.004166666666666667]. All numeric values above are reproduced verbatim from `analysis/results.json` and analysis artifacts.

Bootstrap and test details. The bootstrap CI was computed with 10000 resamples (seed = 1729) using paired resampling of episode-pairs; the analysis executor used a paired-bootstrap-percentile approach (artifact: `analysis/analysis_log.jsonl`). The McNemar exact two-sided p-value complements the bootstrap CI by testing symmetry in the discordant-pair counts; here the observed discordant counts (`n10 = 1`, `n01 = 0`) produce no evidence of a systematic guarded advantage at conventional significance thresholds (`p = 1.0` under the exact test reported by the executor).

Run accounting and discordant-pair provenance. Provider-call accounting and deterministic reconciliation confirm `model_calls_used = 721` (stages: `shared_initial_generation = 720`; `repair = 1`) and `complete_pair_count = 720` (`analysis/deterministic_reconciliation.json`, sha256 49731a2e52e049ccf5b53c35ddac2351ad75e7e95504dc4a42bcb09ed0dec3aa). Because `shared_initial_candidate` semantics were enforced, the single guarded-only detection (the unique `n10` entry) must arise from downstream guarded-stage activity (sandboxed emulation) or from the single recorded repair-stage invocation.

Explicit archival pointer to the discordant episode (artifact-grounded). The unique discordant episode can be located in the archived raw-results file: `execution/raw_results.jsonl` (input-results SHA-256 = df367ecc40a6f0eaf402129260e11d8450884a4ea5fdb94a602e0d2513aef0e8). Within that file, filter for the entry where `baseline_score = 0` and `guarded_score = 1`; that `raw_results.jsonl` record contains the episode's `pair_id` and the linked `validator_trace` and any `repair_provider_trace` fields that document the repair-stage invocation. The deterministic reconciliation artifact (`analysis/deterministic_reconciliation.json`, sha256 49731a2e52e0...) and the `provider_call_audit` block therein corroborate that the repair-stage call recorded in provider accounting maps to that unique discordant pair. We provide these exact artifact identifiers so readers can unambiguously

locate the discordant pair and its repair/emulation traces in the archived bundle (see Reproducibility pointers below).

Discordant attribution and repair linkage. The archived `raw_results` entry for the discordant pair includes `validator_trace` (`validation_before/after`), any emulation observations, and fields capturing whether a repair prompt was issued and its trace path. The `deterministic_reconciliation.json` `provider_call_audit` documents `stage_counts` and confirms exactly one repair-stage call; matching timestamps and trace-path fields in `raw_results.jsonl` link that repair-stage call to the guarded-only detection. Thus the run artifacts substantiate that the single guarded-only detection corresponds to downstream guarded-stage activity (repair/emulation) rather than to generation differences, consistent with `shared_initial_candidate` semantics.

Representative per-episode diagnostics. Representative scoring traces archived under `execution/scoring/task-000001-*.json ... task-000006-*.json` include detailed `validator_trace` objects with `normalized_configuration`, `validation_before/after` states, `parsed_command_count`, `required_command_count`, `violation_count`, and `validator_version = 5.0`. Those representative artifacts illustrate that the vast majority of `shared_initial` candidates complied with validator rules (`violation_count = 0`) and that sandboxed emulation generally corroborated validator findings in this curated task bank. The `analysis/condition_summary.csv` artifact (`sha256 394bc690671b7...`) summarizes successes/failures by condition and reproduces the marginal success rates reported above.

Synthesis of numeric outcomes. In short, the guarded pipeline detected one more successful item than the baseline (720 vs. 719) across 720 paired episodes. The paired estimate (~ 0.00139) and its 95% bootstrap CI ($[0.0, 0.0041667]$) are both orders of magnitude smaller than our preregistered minimum effect-of-interest (0.20). Provider-call accounting (`model_calls_used = 721`, `repair = 1`) further shows that the observed discordance is attributable to recorded downstream activity rather than to between-condition generation variance. These concrete, artifact-grounded outcomes are the basis for the interpretations and operational boundary conditions we discuss elsewhere in the manuscript.

V. DISCUSSION

Why was the guarded-stage aggregate benefit negligible? The archived evidence supports three artifact-grounded factors and suggests practical boundary conditions where guarded stages are—and are not—likely to matter.

1) Validator ceiling effect and task-rule alignment. `Deterministic_netops_validator_v5` explicitly encodes the syntactic and semantic checks exercised by the curated tasks (examples of patterns in task payloads include `shutdown_configure_restore`, `make_before_break_uplink_switchover`, and `paired_link_atomic_mtu_change`). Representative `validator_trace` fields (`parsed_command_count`, `required_command_count`, `violation_count = 0`) in `execution/scoring/task-000001-*.json` show that most

`shared_initial` candidates already satisfied the validator’s rule set. The baseline marginal success rate of 719/720 (0.9986111111111111) leaves essentially no headroom for the guarded emulation stage to flag additional failures in this constrained regime.

2) Curated inclusion and reduced adversarial exposure. The confirmatory sample was assembled from a deterministic generator (`intent_configuration_repair_v1`) and required vendor-parser acceptance before inclusion. This inclusion rule intentionally restricted confirmatory items to parser-validated, policy-constrained scenarios; as a result, many runtime-only faults that emulation detects in more diverse or adversarial settings are underrepresented. The missingness summary and `deterministic_reconciliation` artifacts confirm zero irrecoverable parser exclusions, indicating the set was tightly constrained to inputs the validator could reasonably analyze.

3) `Shared_initial_candidate` semantics and limited repair activity. By design we reused the exact sampled initial candidate for both baseline and guarded conditions; this isolates downstream guarded-stage signals as the only allowable source of between-condition discordance. `Provider_call_audit` and `deterministic_reconciliation` report `model_calls_used = 721` and `stage_counts: shared_initial_generation = 720`, `repair = 1`. The single n10 discordant pair therefore corresponds to the single recorded repair-stage invocation or to a guarded-emulation observation not codified as a validator rule. The `raw_results` and `deterministic_reconciliation` artifacts together allow an auditor to locate that unique `pair_id` and inspect `validator_trace` and `repair_provider_trace` to see whether the repair altered the candidate or whether the emulation exposed a runtime-only violation.

Interpretation and operational implications. For environments that maintain a high-coverage static validator tailored to expected intent patterns, and where inputs are parser-validated and curated, adding an automated sandboxed-emulation stage may have negligible incremental detection benefit at scale. This does not imply sandboxing is unnecessary; rather, it implies a conditional efficiency boundary: when validator coverage closely matches expected task patterns and the input pipeline enforces strong prevalidation, guarded runtime checks will detect relatively few additional faults. Conversely, in operational regimes characterized by (a) noisy natural-language intents, (b) adversarially crafted misconfigurations, (c) broader uncured vendor-CLI diversity, or (d) multi-step stateful interactions, the guarded stage can surface runtime semantics and side-effects that static validators cannot anticipate.

Practical recommendations grounded in the artifacts. The archived execution and analysis artifacts suggest three pragmatic paths depending on operator goals: (i) invest in richer static validators that capture more workflow-level constraints when low-latency, high-throughput intent→config translation is the priority; (ii) deploy guarded sandboxing selectively for high-risk or high-impact changes where emulation fidelity matters; (iii) combine both approaches and use adversarial/fuzzing task banks to identify validator gaps—our evidence package includes representative traces that can seed such

adversarial test-case design.

Limitations of this interpretation rooted in archived evidence. The near-zero aggregate effect we observed is specific to the preregistered task bank, the single validator implementation, the declared/requested model, and the `shared_initial_candidate` semantics. The `execution_manifest` and `deterministic_reconciliation` document the exact run accounting (`model_calls_used` = 721; `repair` = 1) and allow independent verification that the observed discordance is localized to a single downstream event. Because the study was powered a priori for a minimum detectable paired difference of 0.20, the empirical observation (≈ 0.0014) lies far below that sensitivity and must be interpreted as a narrowly scoped near-zero finding for this experimental regime—not as broad evidence that guarded stages are always unnecessary.

Directions where guarded stages are likely to add value. The artifact-grounded diagnostics indicate three concrete extensions where guarded emulation should be retested: (1) adversarially generated tasks specifically designed to exercise validator gaps (e.g., subtle state-dependent ordering errors); (2) expanded vendor-CLI families and syntactic/semantic diversity to challenge parser and validator coverage; (3) higher-fidelity emulation that models vendor-specific control-plane subtleties and long-lived state transitions. The archived per-episode traces in `execution/raw_results.jsonl` and the representative scoring artifacts provide concrete seed cases and validator-rule examples to guide these follow-up efforts.

Concluding synthesis. The run’s provenance and analysis artifacts permit an unambiguous, auditable mapping from the reported aggregate counts to the single discordant episode and the recorded repair-stage call. These artifacts support a principled, evidence-led conclusion: in this preregistered, curated regime, a strong deterministic validator absorbed almost all detectable faults and the guarded sandboxed-emulation stage produced a vanishing marginal detection gain. Whether that conclusion generalizes to more adversarial or production-like regimes requires the targeted extensions described above, for which the current archived bundle provides reproducible starting points.

VI. LIMITATIONS

- Synthetic task bank and deterministic task generator produce limited ecological diversity and create a validator-ceiling effect; results may not generalize to noisier, adversarial, or production distributions.
- Single declared/requested model family (gpt-5-mini) and single validator implementation (`deterministic_netops_validator_v5`) limit generalizability across models and validator families.
- Preregistered `shared_initial_candidate` semantics (one initial generation reused across paired conditions) isolate guarded-stage contribution but remove between-condition generation variability; this design choice constrains discordance sources to downstream guarded-stage observations or repair calls.

- Sandboxed-emulation fidelity relative to vendor/OS production behaviors and large-scale stateful interactions is limited by the emulation harness used; higher-fidelity emulators could change outcomes in other regimes.
- Study power planning targeted a minimum detectable paired difference of 0.20; the observed aggregate effect (~ 0.0014) lies far below that design sensitivity and should be interpreted as a narrowly scoped near-zero finding for this experimental regime rather than a general negative result for all NetOps workflows.
- Provider-side nondeterminism and hidden caching remain residual uncertainties despite deterministic reconciliation procedures; these are documented in `analysis/deterministic_reconciliation.json` and `provider_call_audit` artifacts.

VII. CONCLUSION

We preregistered and executed a paired confirmatory study comparing a strong deterministic validator with the same validator augmented by sandboxed emulation on a curated synthetic intent→configuration task bank. Over 720 paired tasks the guarded pipeline produced a negligible aggregate detection gain (0.0013888888888888884) with exact McNemar $p = 1.0$ and 95% bootstrap CI [0.0, 0.004166666666666667]. Run accounting shows one repair-stage invocation corresponding to the single guarded-only detection; because `shared_initial_candidate` semantics were enforced, archived per-episode traces permit independent verification of that mapping via `execution/raw_results.jsonl` and `analysis/deterministic_reconciliation.json`. We interpret the result as arising from validator ceiling effects and curated task design; follow-up work should focus on adversarial and production-like task banks, multi-validator ensembles, and higher-fidelity emulation to identify regimes where guarded stages add substantive operational value.

Reproducibility pointers (compact). The canonical execution bundle archived by the run contains: `execution/execution_manifest.json` (execution summary and preregistration SHA-256), `execution/raw_results.jsonl` (input-results SHA-256 `df367ecc40a6f0eaf402129260e11d8450884a4ea5fdb94a602e0d2513aef0e8`), `analysis/deterministic_reconciliation.json` (sha256 `49731a2e52e049ccf5b53c35ddac2351ad75e7e95504dc4a42bcb096d0dec3aa`), `execution/model_configuration.json` (sha256 `a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820`), `analysis/paired_contingency_table.csv` (sha256 `efe6e8e7016fa843a8226e911dbde829e943903d719101da8d956b3ff899133c`), and `analysis/condition_summary.csv` (sha256 `394bc690671b731194f9b42b4dcbe28fded6e2ec8cf898b79831225343c22c28`).

The discordant episode is the unique `raw_results.jsonl` entry with `baseline_score` = 0 and `guarded_score` = 1; that record contains the `pair_id` and validator/emulation traces that map it to the single repair-stage invocation recorded in `deterministic_reconciliation.json`. These artifacts are archived as the canonical reproducibility bundle referenced by the

execution manifest and the preregistration record. (See Disclosure for exact manifest references.)

DISCLOSURE STATEMENT

Disclosure: Autonomous post-lock research run executed under the frozen master prompt supplied to the system. Declared/requested model: "gpt-5-mini" (provider recorded as openai_responses). Deterministic validator: deterministic_netops_validator_v5. Orchestration adapter family: hosted_netops_gvr_v1. Preregistration id: f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e (preregistration SHA-256: 0169919fb8c5aedb2c5a78e031f1ab5c3aeda405cde1ae80fb2f864768595c1b; recorded in the execution manifest). Initial master-prompt immutable reference: provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba. Execution summary (artifact-grounded): completed_episode_count = 1440, complete_pair_count = 720, model_calls_used = 721 (720 shared initial + 1 repair-stage call); stage_counts and provider-call audit are archived in analysis/deterministic_reconciliation.json (sha256 49731a2e52e0...). Human role and autonomy boundary: a human Research Director selected the broad NetOps topic family and constrained the pre-lock master prompt; after the autonomy lock the agentic pipeline autonomously performed literature selection, preregistration, code and prompt generation, experiment execution, analysis, peer review, and manuscript drafting. No manual human editing of technical manuscript content occurred after the autonomy lock. The execution manifest and archived artifacts named above serve as the canonical reproducibility bundle.

REFERENCES

- [1] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729
- [2] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866
- [3] <http://arxiv.org/abs/2407.08249>